Введение

Цель работы — развернуть две Linux-виртуальные машины, установить и настроить WireGuard в конфигурации «сервер — клиент», проверить передачу данных через VPN и зафиксировать внешний зашифрованный UDP-трафик в Wireshark.

Для лабораторного стенда использованы две виртуальные машины Lubuntu в Oracle VirtualBox. Адаптер NAT обеспечивал доступ к пакетным репозиториям, а общий Host-only адаптер — связь между узлами. Для туннеля выделена отдельная сеть 10.8.0.0/24.

Узел	Сетевые адреса и роль
Сервер	Host-only: 192.168.56.101 (enp0s8); WireGuard: 10.8.0.1/24; UDP-порт: 51820
Клиент	Host-only: 192.168.56.102 (enp0s8); WireGuard: 10.8.0.2/24; подключение к 192.168.56.101:51820
VPN-сеть	10.8.0.0/24; адрес сервера 10.8.0.1, адрес клиента 10.8.0.2

1. Установка WireGuard

На обеих виртуальных машинах обновлён список пакетов и установлен WireGuard. Команды выполнялись отдельно на сервере и клиенте:

```
sudo apt update
sudo apt install wireguard -y
wg --version
```

Для каждого узла создана собственная пара ключей. Файл приватного ключа создавался с ограниченными правами; приватные значения в отчёте и иллюстрациях скрыты.

```
umask 077
wg genkey | tee ~/wg_private.key | wg pubkey > ~/wg_public.key
```

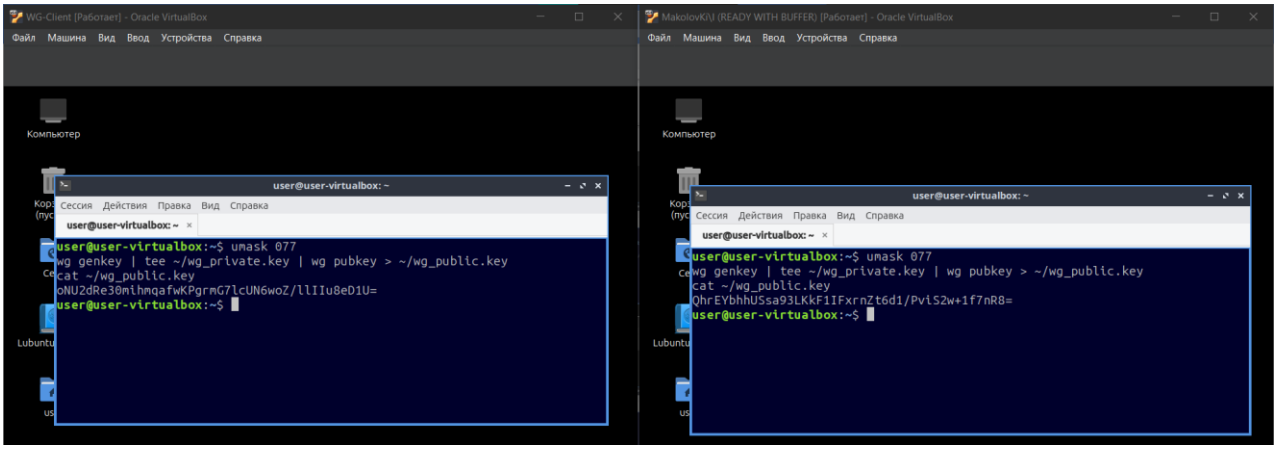


Рисунок 1 — Генерация отдельных пар ключей на сервере и клиенте

2. Настройка сервера WireGuard

На сервере создан файл /etc/wireguard/wg0.conf. Приватный ключ сервера указан локально; в отчёте он заменён обозначением [СКРЫТ]. В поле PublicKey секции [Peer] внесён публичный ключ клиента.

```
[Interface]
Address = 10.8.0.1/24
ListenPort = 51820
PrivateKey = [СКРЫТ]

[Peer]
PublicKey = [публичный ключ клиента]
AllowedIPs = 10.8.0.2/32
```

Конфигурационный файл защищён правами доступа и интерфейс запущен:

```
sudo chmod 600 /etc/wireguard/wg0.conf
sudo wg-quick up wg0
sudo systemctl enable wg-quick@wg0
```

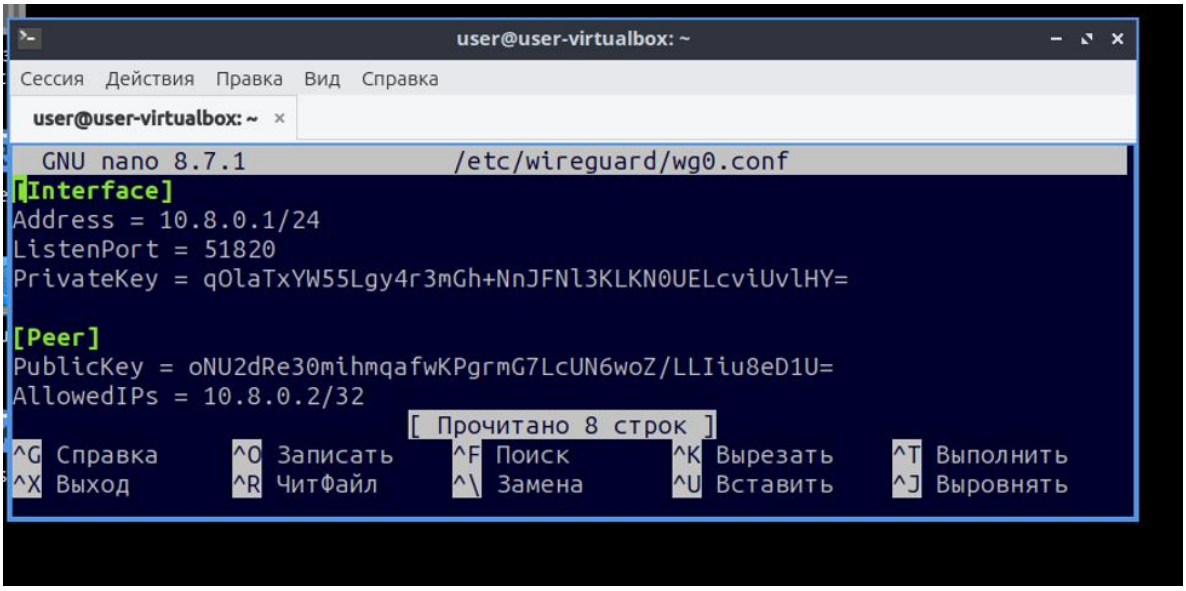


Рисунок 2 — Конфигурация сервера;

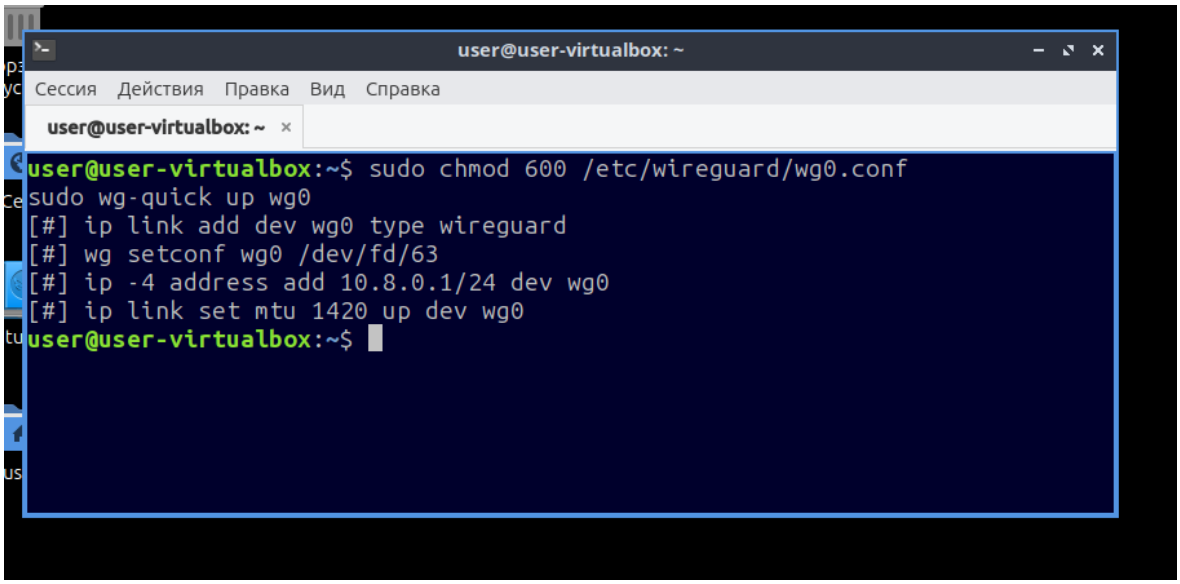


Рисунок 3 — Успешный запуск интерфейса wg0 на сервере

3. Настройка клиента WireGuard

На клиенте настроен интерфейс 10.8.0.2/24. В секции [Peer] задан публичный ключ сервера, адрес его Host-only интерфейса и маршрут только для VPN-сети. Параметр PersistentKeepalive поддерживает связь через NAT.

```
[Interface]
Address = 10.8.0.2/24
PrivateKey = [СКРЫТ]

[Peer]
PublicKey = [публичный ключ сервера]
Endpoint = 192.168.56.101:51820
AllowedIPs = 10.8.0.0/24
PersistentKeepalive = 25
```

После установки конфигурации ограничены права доступа к файлу и поднят интерфейс wg0:

```
sudo chmod 600 /etc/wireguard/wg0.conf
sudo wg-quick up wg0
sudo systemctl enable wg-quick@wg0
```

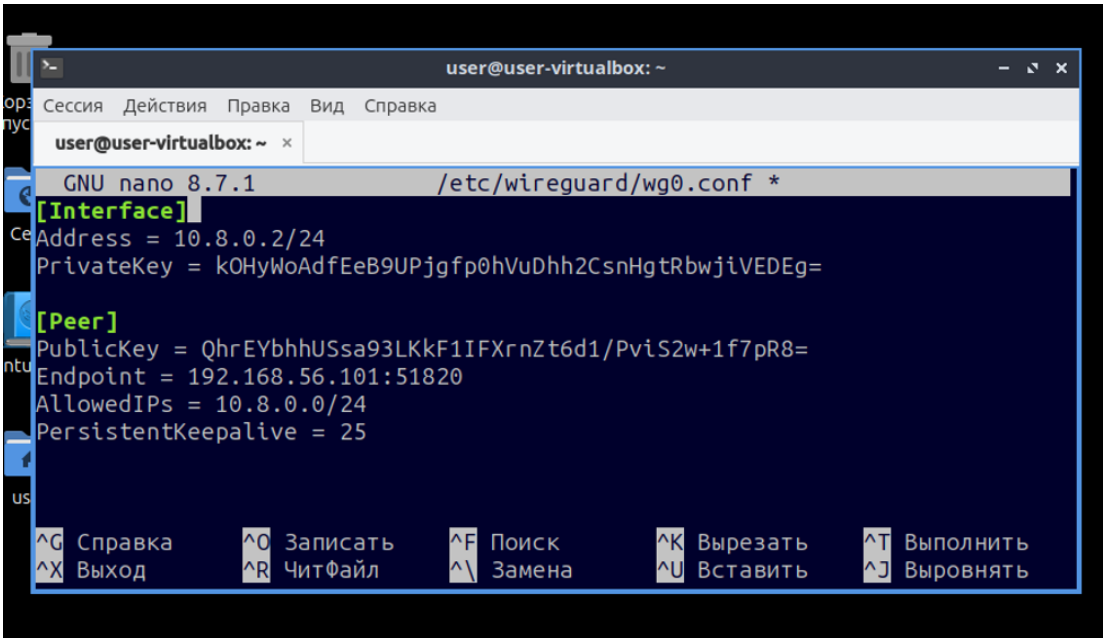


Рисунок 4 — Конфигурация клиента;

4. Проверка работы VPN

После запуска туннеля связь проверена ICMP-запросами от клиента к VPN-адресу сервера:

ping -c 4 10.8.0.1

Получены четыре ответа из четырёх, потерь пакетов нет. Вывод sudo wg show на клиенте показывает установленный handshake и ненулевые счётчики принятых и отправленных данных.

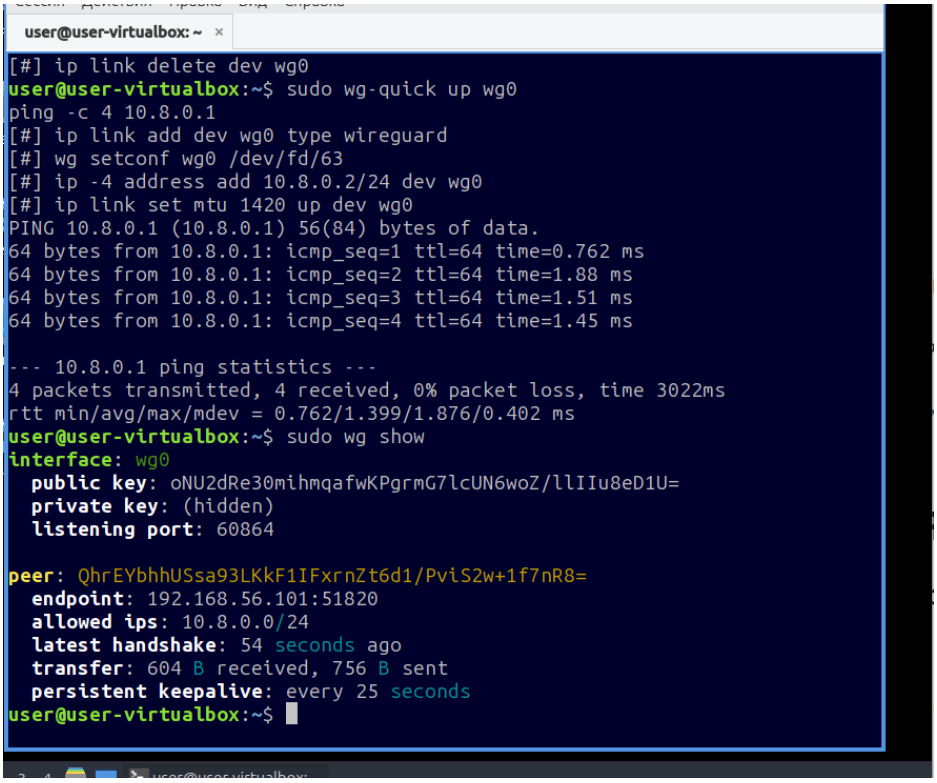


Рисунок 5 — Успешный ping через туннель и состояние WireGuard (latest handshake, transfer)

5. Демонстрация трафика в Wireshark

Для наблюдения выбран физический интерфейс enp0s8 на сервере. В Wireshark применён фильтр отображения udp.port == 51820; во время захвата на клиенте выполнялся ping VPN-адреса сервера.

В списке пакетов виден обмен между 192.168.56.102 и 192.168.56.101 по UDP, декодируемый Wireshark как WireGuard Transport Data. Это подтверждает передачу VPN-трафика по внешней сети; содержимое туннеля не отображается в открытом виде.

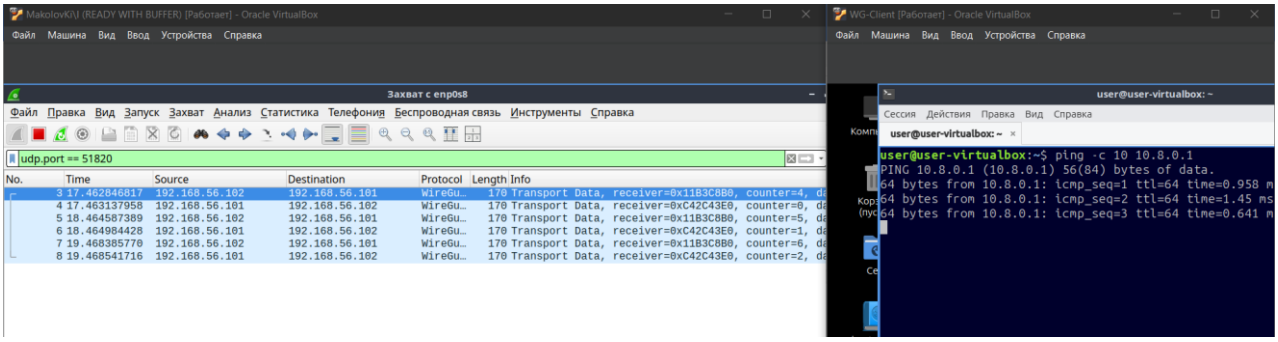


Рисунок 6 — Пакеты WireGuard в Wireshark и ping через VPN

6. Выводы и сложности

В ходе работы WireGuard установлен и настроен на двух Linux-виртуальных машинах. Клиент установил соединение с сервером, передал данные по туннелю 10.8.0.0/24, а успешный обмен подтверждён ping, состоянием handshake в wg show и захватом UDP-пакетов в Wireshark.

При первой проверке VPN-туннель не отвечал: причиной было несовпадение публичных ключей, внесённых в секции [Peer]. После сверки ключей с выводом wg show и обновления конфигураций handshake установился, ping стал успешным. Этот этап показал важность точной проверки ключей и адресов узлов.